

Modular Synthesis of Sketches Using Models

Rohit Singh, Rishabh Singh, Zhilei Xu, Rebecca Krosnick,
and Armando Solar-Lezama*

Massachusetts Institute of Technology

Abstract. One problem with the constraint-based approaches to synthesis that have become popular over the last few years is that they only scale to relatively small routines, on the order of a few dozen lines of code. This paper presents a mechanism for modular reasoning that allows us to break larger synthesis problems into small manageable pieces. The approach builds on previous work in the verification community of using high-level specifications and partially interpreted functions (we call them models) in place of more complex pieces of code in order to make the analysis modular.

The main contribution of this paper is to show how to combine these techniques with the counterexample guided synthesis approaches used to efficiently solve synthesis problems. Specifically, we show two new algorithms; one to efficiently synthesize functions that use models, and another one to synthesize functions while ensuring that the behavior of the resulting function will be in the set of behaviors allowed by the model. We have implemented our approach on top of the open-source Sketch synthesis system, and we demonstrate its effectiveness on several Sketch benchmark problems.

1 Introduction

Over the last few years, constraint-based approaches to synthesis based on sketches or templates have become quite popular [11,19,26,28,29]. In these approaches, the user specifies her intent with a template of the desired solution leaving parts of the code unspecified (a sketch), and the synthesizer finds the unknown code fragments such that the completed template conforms to a given set of behavioral constraints (the spec). One problem with such approaches, however, is that they only scale to relatively simple routines, on the order of a few dozen lines of code. Scaling such methods to more complex programs requires a mechanism for modular reasoning.

The idea of modular reasoning has been quite successful and is widely used today in verification of software and hardware, where pre and post-conditions are commonly used to model complex functions (*e.g.* MAGIC [7], DAFNY [14]), and where uninterpreted or partially interpreted functions play an important role in abstracting away complex functional units [4,5]. These assume-guarantee

* This work was funded by NSF grants NSF-1116362, NSF-1139056, and DOE grant DE-SC0005372.

reasoning based approaches perform compositional verification by breaking down the verification task of a system into smaller tasks that involve verification of individual components, which enables the verification tools to then compose the proofs to verify the whole system [18,30].

In this paper, we present a mechanism of modular reasoning for synthesizing complex sketches, where we use *function models* to specify the behavior of constituent function components. A function model consists of three components: pre-processing code that canonicalizes the input, an uninterpreted function that models the function behavior, and a post-condition that specifies the desired properties of a function. A model can also have a pre-condition that specifies which parameters are legal for the function. However, we place two restriction on models: 1) they cannot have any unknown code fragment (holes) and 2) they cannot post-process the output of the uninterpreted function before returning it. These function models are more general than pre and post-conditions without uninterpreted functions, but are less general than pre and post-conditions with quantifiers. This intermediate generality of function models provides us the expressiveness to specify many complex functions and at the same time allows us to efficiently use them for synthesis.

The function models in sketches introduce two new synthesis problems: 1) synthesis with models and 2) synthesis for adherence. The synthesis with models problem requires us to solve for unknown control parameters such that for any uninterpreted function that satisfies the model's post-condition, the specification of main function should also hold. This problem in principle can be solved by the traditional CEGIS [24] algorithm but CEGIS performs poorly in practice. We present a new algorithm CEGIS+ that combines the CEGIS algorithm with an approach that selectively uses existential quantification for some inputs [11] for efficiently solving this problem. The sketch will use the function model in place of a more complex function that itself may have unknowns. Therefore, we also need to solve a second synthesis problem: synthesis for adherence, which ensures that this more complex function is synthesized in a way that matches the behavior promised by the model. This synthesis problem introduces a standard doubly quantified constraint but with an existentially quantified function to find a function that satisfies the model post-condition on all valid inputs and has an equivalent input-output relationship with the original function. To solve this problem, we present an approach to eliminate the existentially quantified function by taking advantage of the specific form of the constraint and do not require additional function templates unlike previous approaches [24,33].

We have implemented the algorithms in the SKETCH synthesis system [25] and present the evaluation of the algorithm on several benchmark problems. We show that function models enable synthesis of several complex benchmarks, and our algorithm outperforms both CEGIS and previously published algorithms.

Specifically, the paper makes the following contributions:

- We show that the existing approaches to counterexample guided synthesis break down in the presence of models and present a new algorithm that can efficiently synthesize functions that use models.